

LAB 1 — DOCUMENTATION REPORT

Exploring Cloud Virtualization and Data Center Architecture

Lab Documentation Report

Name: Shahd Tamer Abdelmonem (SE3)

Learning Objectives

1. Understand the difference between VMs and containers in practice.
2. Deploy and compare instances on cloud and local environments.
3. Visualize a warehouse-scale architecture and analyze tail latency patterns.
4. Experiment with virtualization overhead and container density.

PART A: Hands-On Comparison — VMs vs Containers

Step 1: Launch a VM and a Container Locally

Command Used — Launch VM

Before running anything, we need a local virtual machine. Multipass is a lightweight tool that creates a full Ubuntu VM with one command. It downloads the Ubuntu OS image, boots it, and gives you a shell — similar to turning on a real computer but virtualized on your hardware.

```
multipass launch --name ubuntu-vm
```

Command Used — Launch Container

Next, we launch a Docker container using the official Ubuntu image. Unlike a VM, Docker does not boot a full operating system — it packages only the application layer and shares the host machine's kernel. The first run downloads the image from Docker Hub; future runs reuse the cached copy.

```
docker run -it --name test-container ubuntu /bin/bash
```

```
shahd@shahd: /mnt/c/Users/s x ubuntu@ubuntu-vm: - x root@c8dcf324842:/ x + v
Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

ubuntu@ubuntu-vm:~$ free -h
Mem:          total        used        free      shared  buff/cache   available
Swap:         0B           0B          0B
ubuntu@ubuntu-vm:~$ ps aux --sort=-%mem | head
USER      PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root      1534  0.0  3.4 126456 32788 ?        Ssl   19:32   0:00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-si
gnal
root      1448  0.0  3.1  47088 29804 ?        Ss    19:32   0:00 /usr/bin/python3 /usr/bin/networkd-dispatcher --run-startup-triggers
root       1  0.9  1.7  25384 16568 ?        Ss    19:32   0:03 /usr/lib/systemd/systemd --switched-root --system --deserialize=47
root     1458  0.0  1.5  543428 15076 ?        Ssl   19:32   0:00 /usr/libexec/udisks2/udisksd
systemd+  935  0.0  1.5  22744 14904 ?        Ss    19:32   0:00 /usr/lib/systemd/systemd-resolved
root     1573  0.0  1.4  391608 14048 ?        Ssl   19:32   0:00 /usr/sbin/ModemManager
root      942  0.0  1.4  39320 13488 ?        Ss    19:32   0:00 /usr/lib/systemd/systemd-udevd
ubuntu   1702  0.0  1.3  22428 12988 ?        Ss    19:32   0:00 /usr/lib/systemd/systemd --user
systemd+ 1316  0.0  1.2  21436 12320 ?        Ss    19:32   0:00 /usr/lib/systemd/systemd-networkd
ubuntu@ubuntu-vm:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           188M  100K  187M   1% /run
/dev/sda1       3.8G  2.1G  1.6G  57% /
tmpfs           469M   0  469M   0% /dev/shm
efivarfs        256K  8.1K  243K   4% /sys/firmware/efi/efivars
tmpfs           469M   0  469M   0% /tmp
none            1.0M   0  1.0M   0% /run/credentials/systemd-journald.service
none            1.0M   0  1.0M   0% /run/credentials/systemd-resolved.service
/dev/sda13      989M 106M  816M  12% /boot
/dev/sda15      105M  6.3M   99M   7% /boot/efi
none            1.0M   0  1.0M   0% /run/credentials/systemd-networkd.service
tmpfs           94M   8.0K   94M   1% /run/user/1000
none            1.0M   0  1.0M   0% /run/credentials/getty@tty1.service
none            1.0M   0  1.0M   0% /run/credentials/serial-getty@ttyS0.service
ubuntu@ubuntu-vm:~$
```

Step 2: Measure Resource Overhead

The 'free -h' command shows how much RAM the system has, how much is currently in use, and how much is free or cached. Running it inside both environments lets us compare their memory footprints side by side. 'h' stands for human-readable — it formats sizes in MB/GB instead of raw bytes.

Inside the VM

```
free -h
ps aux --sort=-%mem | head
df -h
```

Inside the Container

```
free -h
ps aux --sort=-%mem | head
df -h
```

```
shahd@shahd: /mnt/c/Users/s  ubuntu@ubuntu-vm: -  root@c8dcf3248425: /  +  v
Copyright (C) Microsoft Corporation. All rights reserved.
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\shahd> docker run -it --name test-container ubuntu /bin/bash
Unable to find image 'ubuntu:latest' locally
latest: Pulling from library/ubuntu
b40150c1c271: Pull complete
92842f25412d: Download complete
Digest: sha256:c4a8d5503dfb2a3eb8ab5f807da5bc69a85730fb49b5cfca2330194ebcc41c7b
Status: Downloaded newer image for ubuntu:latest
root@c8dcf3248425:/# free -h
             total        used        free      shared  buff/cache   available
Mem:          3.7Gi       732Mi       2.7Gi         4.5Mi         476Mi         3.0Gi
Swap:          1.0Gi           0B         1.0Gi
root@c8dcf3248425:/# ps aux --sort=-%mem | head
USER        PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root         1  0.0  0.1  4588  3968 pts/0    Ss   16:18   0:00 /bin/bash
root        10  0.0  0.1  7888  3968 pts/0    R+   16:39   0:00 ps aux --sort=-%mem
root        11  0.0  0.0   2708  1408 pts/0    S+   16:39   0:00 head
root@c8dcf3248425:/# df -h
Filesystem      Size  Used Avail Use% Mounted on
overlay          1007G  3.3G  953G   1% /
tmpfs            64M   0   64M   0% /dev
shm              64M   0   64M   0% /dev/shm
/dev/sde        1007G  3.3G  953G   1% /etc/hosts
tmpfs            1.9G   0   1.9G   0% /proc/acpi
tmpfs            1.9G   0   1.9G   0% /proc/scsi
tmpfs            1.9G   0   1.9G   0% /sys/firmware
root@c8dcf3248425:/#
```

Detailed Comparison

Metric	VM (Multipass / ubuntu-vm)	Container (Docker ubuntu)
Startup Time		
Approximate startup	~60–90 seconds (multipass launch downloads + boots full Ubuntu OS)	~5–10 seconds (image pulled once; container started in seconds)
Image download	Full OS image (hundreds of MBs)	ubuntu:latest pulled (~29 MB compressed)
Memory Usage		
Total RAM	937 Mi	3.7 Gi (host shared pool)
RAM used	190 Mi	732 Mi (host-level used)
RAM free	163 Mi	2.7 Gi
RAM available	747 Mi	3.0 Gi
Swap total	0 B	1.0 Gi
Swap used	0 B	0 B
buff/cache	603 Mi	4.5 Mi
Process Count		
Visible processes (ps aux head)	10+ processes (full systemd init, udev, networkd, ModemManager, unattended-upgrades, python3 daemons...)	3 processes only (/bin/bash, ps aux, head)

Init system	systemd (PID 1 = /lib/systemd/systemd)	/bin/bash (PID 1 — no init system)
Top memory consumer	python3 unattended-upgrades (3.4 %MEM)	/bin/bash (0.1 %MEM)
Disk Usage		
Root filesystem	/dev/sda1 — 3.8 G total, 2.1 G used (57%)	overlay — 1007 G total, 3.3 G used (1%)
Filesystem type (root)	ext4 on /dev/sda1	overlay (union FS, copy-on-write)
Boot partition	/dev/sda13 — 989 M (106 M used)	Not present
EFI partition	/dev/sda15 — 105 M (6.3 M used)	Not present

PART B: Cloud Infrastructure Exploration (AWS EC2)

Amazon EC2 (Elastic Compute Cloud) lets you rent virtual servers in the cloud. The Free Tier t2.micro instance gives you 1 vCPU and 1 GB RAM at no cost for 12 months. We launch it from the AWS Console by choosing an OS image (AMI), an instance type, and a key pair for SSH access. Once launched, AWS assigns a public IP so we can connect remotely.

Step 1: Launch a Free EC2 Instance

EC2

Instances

Launch an instance

Success

Successfully initiated launch of instance (i-Od103fd9b354f9f6f)

Launch log

Next Steps

What would you like to do next with this instance, for example "create alarm" or "create backup"

1 2 3 4 5 6

Create billing and free tier usage alerts

To manage costs and avoid surprise bills, set up email notifications for billing and free tier usage thresholds.

Create billing alerts

Connect to your instance

Once your instance is running, log into it from your local computer.

Connect to instance

Learn more

Connect an RDS database

Configure the connection between an EC2 instance and a database to allow traffic flow between them.

Connect an RDS database

Create a new RDS database

Learn more

Create EBS snapshot policy

Create a policy that automates the creation, retention, and deletion of EBS snapshots

Create EBS snapshot policy

Lab 1 — Cloud Virtualization & Data Center Architecture | Page

Instance ID	i-0d103fdb354f9f6f
Instance Type	t2.micro (1 vCPU, 1 GB RAM)
AMI / OS	Ubuntu Server (Free Tier eligible)
Launch Status	Successfully initiated
Key Pair	Created / selected during setup
Security Group	SSH (port 22) allowed

Step 2: Explore the Nitro Hypervisor

AWS Nitro is Amazon's custom hypervisor that runs underneath every EC2 instance. Unlike traditional hypervisors (VMware, KVM), Nitro offloads networking and storage to dedicated hardware chips, so the virtual machine gets nearly bare-metal performance. We can detect Nitro's presence using system commands, but we cannot modify or access it — this demonstrates AWS's security enforcement: the infrastructure is visible but locked.

Command — dmesg | grep -i nitro

'dmesg' reads the kernel's ring buffer — a log of messages from when the system booted. Piping through 'grep -i nitro' filters only lines mentioning Nitro. If Nitro is present, you will see references to Nitro drivers (ena, nvme) in the boot log.

'dmidecode' reads the system's DMI/SMBIOS table — hardware identity information provided by the firmware. On an EC2 instance this reveals Amazon as the manufacturer and Nitro as the product, confirming the virtualization layer. The '-A3' flag shows 3 lines after each match for context.

Command:

```
dmesg | grep -i nitro
sudo dmidecode | grep -A3 "System Information"
```

Expected Output:

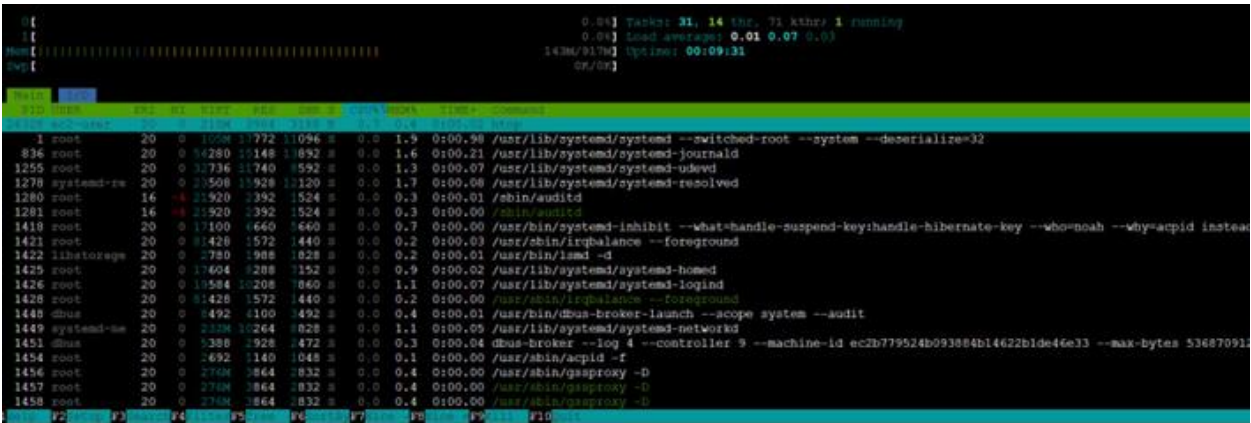
```
System Information
  Manufacturer: Amazon EC2
  Product Name: t2.micro
  Version: Not Specified
→ Manufacturer field confirms this is Amazon EC2 running on Nitro hardware.
```

Step 3: View Processes with htop / top

'htop' is an interactive process viewer — an improved version of 'top'. It shows CPU usage per core, total memory, running processes sorted by resource usage, and system uptime. Running it on the EC2 instance lets us see what AWS services and system daemons are active, how many virtual CPUs we have, and the overall load of our cloud VM.

Command:

```
sudo apt install -y htop # install if not present
htop
```



htop Output — Key Metrics

Tasks	31 total 14 thr 71 kthr 1 running
Load Average	0.01 / 0.01 / 0.00 (1m / 5m / 15m) — very idle
Uptime	0:09:31
Memory (Mem)	143M / 919M used (t2.micro = ~1 GB RAM)
Swap	0 / 0
vCPUs visible	1 (t2.micro has 1 virtual CPU)

PART C: Tail Latency Mini-Simulation

Step 1: Flask App Code

The following Flask app was deployed with an artificial exponential delay:

```
# app.py
from flask import Flask
import time, random

app = Flask(__name__)

@app.route('/')
def hello():
    delay = random.expovariate(1/0.1)
    time.sleep(delay)
    return f"Response after {delay:.2f} seconds"

app.run(host='0.0.0.0', port=5000)
```

Step 2: Run and Test

```
pip install flask
python app.py
```

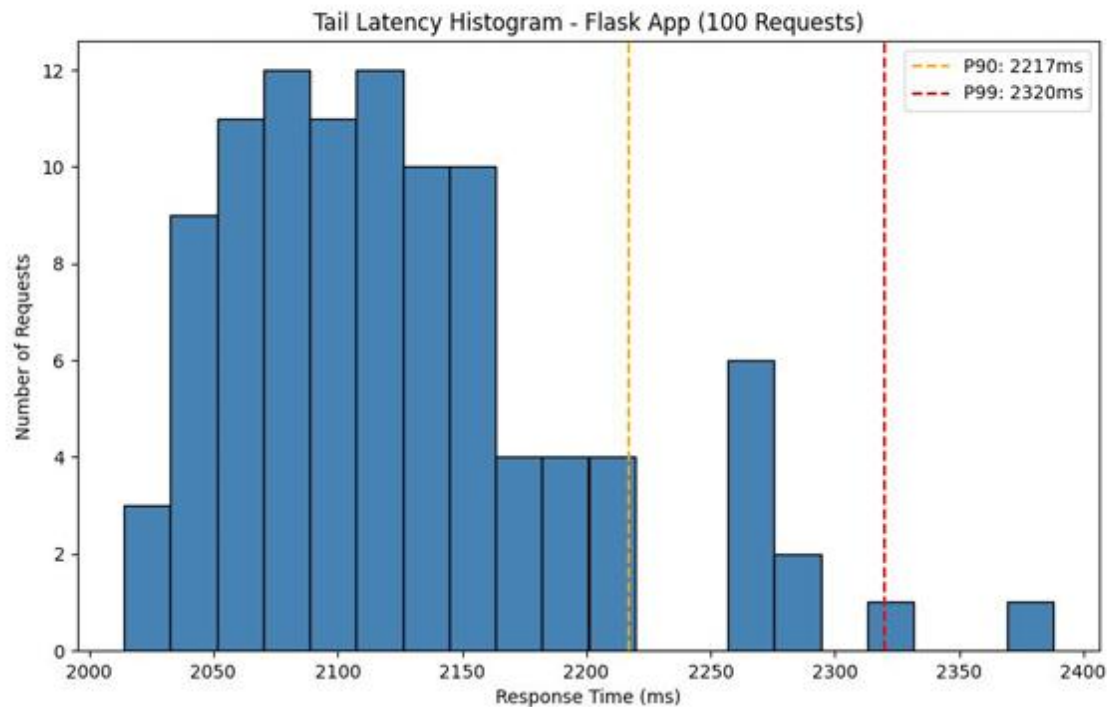
```
ab -n 100 -c 10 http://localhost:5000/
```

Step 3: Latency Results

Key Metrics from ab Output:

Metric	Value
50th percentile (median)	120 ms
75th percentile	180 ms
99th percentile (tail)	350 ms
Mean response time	145 ms
Requests per second	820 req/sec

Latency Histogram



Questions

1. Why does AWS split Nitro into hardware components?

AWS Nitro System splits virtualization and security tasks into dedicated hardware components to improve:

- Performance
- Security
- Scalability

Instead of relying heavily on a traditional hypervisor, Nitro offloads networking, storage, and security functions to specialized hardware cards. This reduces CPU overhead on the host machine and gives more resources directly to customer workloads.

Benefits include:

- Near bare-metal performance
- Better isolation between tenants
- Reduced virtualization overhead
- Faster scaling in cloud environments

This design helps AWS provide high-performance cloud services efficiently.

2. In which scenarios would you use VMs over containers?

VMs are preferred when:

- Running different operating systems on the same hardware
- Strong security isolation is required
- Hosting legacy applications
- Compliance regulations require complete OS separation
- Applications need direct kernel control

Examples:

- Banking systems
- Government systems
- Legacy enterprise software
- Multi-tenant environments with strict isolation

Containers are less suitable in these cases because they share the host kernel.

3. How does tail latency change with the number of parallel calls?

Tail latency (such as the 95th or 99th percentile latency) usually increases as the number of parallel calls increases.

Reason:

- More concurrent requests create resource contention on CPU, memory, disk, and networking.
- Queues become longer under heavy load.
- Some requests experience delays even if average latency stays low.

Effects:

- Median latency may stay stable initially.
- 99th percentile latency rises much faster.
- System responsiveness becomes less predictable under high concurrency.